Deterministic Limit Order Book & Trading System

Review handbook: what was built, why it is designed this way, and how we know it is right

---

Scope: L1 core matching · L2 performance engineering · L3′ market data & pipeline · L4 order entry (L5 and L6 are future plans)
Repository: main ee03bd5 (2026-11-05) · generated 2026-11-05
Every number comes from docs/handbook/numbers.yaml, with its source; † marks wall-clock numbers.
The Chinese edition (handbook-zh.pdf) uses the same outline and section numbers.

Contents

0 How to use this handbook 3
1 The project on one page 4
2 L1 core matching engine 5
3 L2 performance engineering 7
4 L3′ market data and pipeline 8
5 Key story: the resume grace (D-054 / D-055) 10
6 Market-wide mode and real data 12
7 L4 order entry and tick-to-trade 12
8 Engineering method 15
9 Limitations and unverified items (with how to answer) 15
10 Selected interview questions 16
11 Future plans: L5 and L6 (not implemented) 17
A Wall-clock numbers at a glance 18
B Re-measure locally and regenerate 19

How to read: chapters 0 and 1 give the two-minute story; then pick chapters by role. Chapters 9 and 10 are interview material. The footer explains †.

0 How to use this handbook

This handbook covers every level the ROADMAP completed: **L1 core matching**, **L2 performance engineering**, **L3′ market data and pipeline**, and **L4 order entry**. L5 and L6 are future plans (chapter 11). Each chapter follows the same order: **what I built → why it is designed this way → how I know it is right**. The boxed paragraphs are written to be said out loud.

> **The 30-second version**
>
> I built a deterministic limit order book and matching engine from scratch in C++20. On top of it I added a Nasdaq ITCH market-data feed, a multi-threaded pipeline, a strategy with pre-trade risk checks, and finally order entry over OUCH and SoupBinTCP to a single-threaded epoll exchange simulator. The guiding rule was correctness first, then verifiability, then speed. Every layer is checked against a deliberately simple reference model, the multi-threaded and networked modes must match the single-threaded, in-process results bit for bit, and every performance number says where it was measured.

> **The 2-minute version (tell it in this order)**
>
> 1. **L1**: a price-time priority matching engine. Every check happens before anything is modified, and the event stream alone can rebuild the state. I ran 20 million commands of differential testing against a reference model, and the journal and snapshots survive the process being killed.
> 2. **L2**: I wrote the measurement plan before measuring. Profiling showed the session layer copied its whole history on every commit. After switching to geometric growth, same-session alternating runs went from 4.3–4.6 k commands/s† to 0.71–1.09 M commands/s†, and I'm careful to call that a complexity bug fix, not "the engine got hundreds of times faster".
> 3. **L3′**: ITCH 5.0 decoding, order-by-order book reconstruction, and a lock-free SPSC queue between the decoder and book threads. The multi-threaded output is byte-identical to single-threaded replay. I validated it on full days of real Nasdaq data, and **the real data broke one of my rules** (chapter 5).
> 4. **L4**: order entry over OUCH 4.2 with SoupBinTCP sessions, reconnect handling and duplicate-order protection. In lockstep mode the network path matches the in-process simulation bit for bit. The software path to the first order takes 0.37 µs†; what loopback adds depends on the machine. On the cloud VM it was mostly waking up the other thread, but when I re-measured on my own machine that didn't hold, so I kept both results.

Suggested reading: chapters 0 and 1 first; chapters 5, 9 and 10 the day before an interview. Numbers marked † are wall-clock numbers (throughput, latency, tick-to-trade); the page footer says what † currently means. One command in Appendix B re-measures them on your machine and regenerates this handbook.

1 The project on one page

1.1 What I built

Level	Content	Evidence in one line
L1 core	types and validation, order book, matching engine, test infrastructure, replay / snapshot / recovery	197 tests per preset; 20 million commands of differential soak; all 25 mutants killed
L2 performance	measurement method, profiling, one hypothesis-driven experiment (E1), baseline switch, equivalence, memory	same-session alternating A/B; 16 inputs byte-identical across both implementations
L3′ market data	ITCH 5.0 decoding, book reconstruction, feed state machine, SPSC pipeline, strategy and risk, market-wide mode	single-threaded == multi-threaded; real data: 26,040 stock-days over three trading days, all Live
L4 order entry	OUCH 4.2, SoupBinTCP, order gateway, epoll exchange simulator, tick-to-trade	lockstep network mode bit-identical to in-process; free-run recordings replay bit for bit

Across the repository: 6 build presets (GCC / Clang × Debug / Release / ASan+UBSan) with 501 tests each and zero warnings, plus two TSan presets with 79 concurrency tests each; decisions D-001 to D-065 are written down.

1.2 Overall architecture

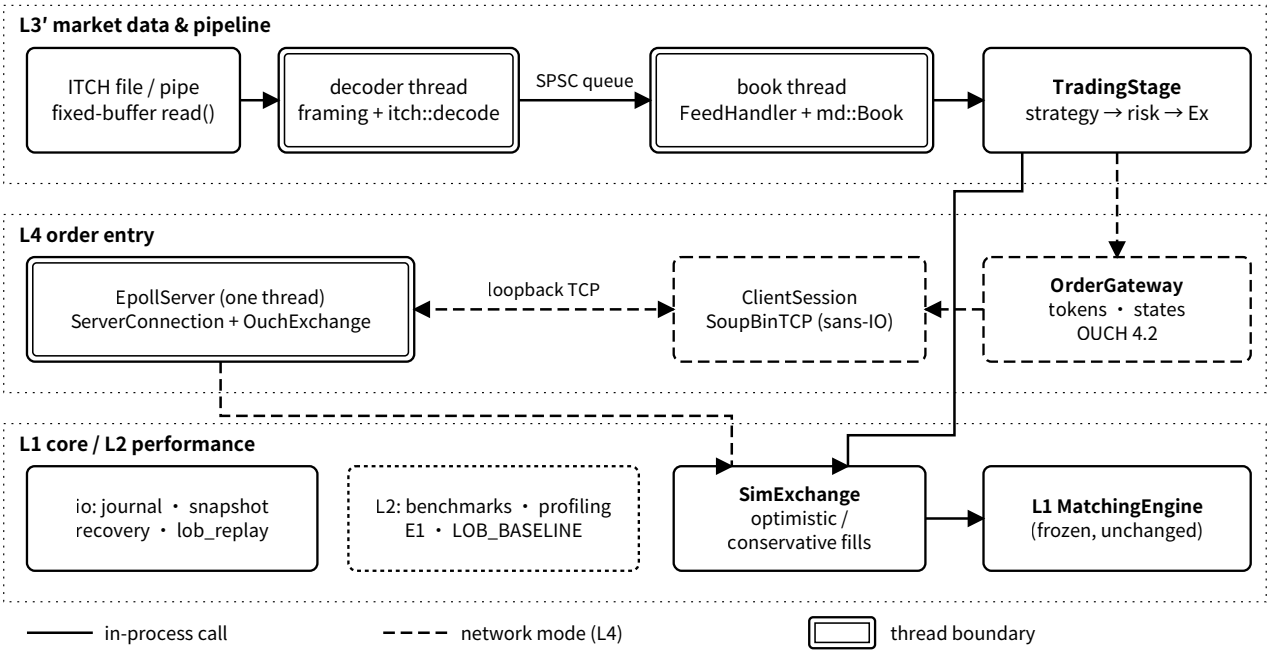


Figure 1. Overall architecture. The three bands are L3′, L4 and L1/L2; double borders are separate threads, dashed lines are the L4 network mode.

Data comes in at the top left. The decoder thread decodes ITCH frames from a file or a pipe and hands them over an SPSC queue to the book thread. The book thread rebuilds the order book, produces market events and passes them to the trading stage: strategy, then risk, then the exchange. There are two exchanges: the in-process `SimExchange`, which wraps the frozen

† wall-clock number: partly re-measured locally, origin in App. A

L1 matching engine, or the path through the order gateway, OUCH and SoupBinTCP over loopback TCP to a single-threaded epoll exchange simulator — which runs the very same `SimExchange` inside.

1.3 Three principles that run through the project

- **Correctness, then verifiability, then performance.** No optimization is worth it if I can no longer prove the output didn't change, and a performance number only counts if it states its environment and method.
- **Determinism.** Prices are integer ticks; the risk token bucket refills on market time, not system time; the multi-threaded and networked modes must produce exactly the same bytes as single-threaded, in-process runs. Determinism is what makes cross-checking possible, and cross-checking is what makes correctness provable.
- **Measure before concluding, and write down what isn't verified.** Every wall-clock number says which machine it came from; the cloud finding "loopback is mostly wake-up cost" didn't reproduce on my own machine, so both results are kept; power-loss durability is "not verified"; the L1 engine still allocates when an order rests, so "zero allocation" is only claimed for the market-data hot path.

2 L1 core matching engine

2.1 Types and error handling

- **Prices are `int64` integer ticks** (D-001): floating point can't compare exactly and its sums aren't deterministic. **Quantities are signed `int64`** (D-003): with unsigned types a bug in `remaining - fill` silently wraps to a huge positive number; with signed types validation and UBSan catch it.
- **Bad input returns a reject code; bad configuration throws** (D-004). An invalid order is normal traffic, not an exception, so the hot path never throws or allocates; a bad configuration is a deployment error and fails at construction.
- **Configuration limits rule out overflow** (D-006): `max_live_orders × max_order_quantity ≤ INT64_MAX`, so no running sum needs a per-step overflow check — overflow is excluded when the engine is configured.

2.2 The order book (M2)

The baseline uses standard containers: a `std::map` of price levels per side, a `std::list` per level for FIFO order, and an `unordered_map` from order id to position (D-007). I built a baseline that is easy to prove correct first, and kept intrusive lists or object pools as L2 experiments rather than adding them up front. The book **contains no matching logic**; it only stores resting orders in price-time priority. In tests, `find_invariant_violation()` re-derives every structural fact after every step: level totals, FIFO order, index consistency, never crossed.

2.3 The matching engine (M3)

- **Price-time priority**, trading at the maker's price; one order can sweep several levels.
- **One sequence clock** (D-020): every event, rejections included, takes the next sequence number, and a resting order's time priority is the sequence number of its acceptance. One

† wall-clock number: partly re-measured locally, origin in App. A

clock gives both the total order of events and the queue order, so an auditor can read the queue straight off the event stream.

- **Two phases** (D-019): the prepare phase only reads — all validation, the sequence overflow check, reserving room for the events. The commit phase neither allocates nor throws; the only possible failure is allocating the resting remainder, and then the engine emits a terminating event before rethrowing (D-024), so the event stream is always complete.
- **When to reject with `BOOK_FULL`** (D-021): simulate the match first and reject only if a remainder would rest in a full book. One can prove that a rejected order would not have traded at all, so there is never a "half filled, then rejected" side effect.

2.4 Test infrastructure (M4)

The main evidence for correctness isn't hand-written cases but **differential testing**: a reference engine written to be obviously correct runs the same random command stream as the real engine, and every event and snapshot is compared.

- Soak matrix: 10 workload profiles, 20 million commands in total, with a full comparison every 1,000 commands. The L1 gate asked for $\geqslant$ 20 seeds $\times$ $\geqslant$ 10,000 commands.
- An event auditor independently checks conservation from the event stream (accepted = 2 $\times$ traded + resting + cancelled), summing in 128-bit integers so the auditor itself can't overflow.
- Fault injection: the global `operator new` is replaced so that the k-th allocation fails, covering every allocation point; in long runs 2% of the commands are armed and the run keeps going.
- Mutation testing: I deliberately broke the engine — "trade at the taker's price", "BOOK_FULL ignores freed slots" — to confirm the tests catch it: all 25 mutants killed.
- Cross-compiler: GCC and Clang produce identical coverage, state and fault lines.

2.5 Replay, snapshots and recovery (M5)

- **Text format v1** (D-034): strict, canonical and versioned. It rejects extra spaces and leading zeros, so "byte-identical" means "semantically identical", which makes golden files possible (5 hand-written scenarios plus one fixed-seed 2,000-command scenario).
- **The journal is one checksummed batch per command, and the events are the source of truth** (D-035): recovery folds events rather than re-executing commands, so it is exact even when the history contains allocation failures.
- **Torn tail versus corruption** (D-036): a half-written tail is dropped and truncated; a failed checksum with more data after it is corruption and blocks automatic recovery. Making five kinds of modification to every byte of the last batch gave 417 corruption reports, 4 dropped batches and **zero** accepted modifications.
- **Commit protocol** (D-038): persist first, then acknowledge; a persistence failure "poisons" the session and only recovery from the journal can continue. A real process killed with `SIGKILL` and restarted ends in the same state as an uninterrupted run.

† wall-clock number: partly re-measured locally, origin in App. A

6 / 19

2.6 How I know L1 is right

Property	Evidence
matching semantics	at least one named test per rule: price priority, FIFO at a price, partial fills, multi-level sweeps, every cancel case, every reject reason, boundary values
agrees with the reference model	20 million commands of differential soak; every CI build also runs 20 seeds × 5,000 commands
determinism	replaying twice, and GCC vs Clang builds, give identical event-stream and snapshot hashes; golden files match byte for byte
exception safety	fault injection at every allocation point; soak runs under ASan / UBSan / LSan
crash consistency	truncation at every byte, injected write failures, a real `kill -9` and restart

3 L2 performance engineering

3.1 Method: write the plan, then measure

- The plan was fixed before any measurement (BENCHMARK §1–§6): data generation and validation stay outside the timed region; throughput and per-operation latency are measured separately; every repetition starts from the same state and checks the state hash.
- **Conclusions only from alternating A/B runs in the same session** (D-043). If the ranges of the processes overlap, the difference is noise; in this environment anything within ±10% can't be told apart. Numbers from two sessions once differed by 2–23% with identical code — that is exactly why the rule exists.
- The author's WSL2 VM has no hardware performance counters, so cache and branch behaviour only comes from cachegrind simulation, always labelled as simulated.

On the seven workload profiles the engine handles millions to tens of millions of commands per second with p99 around 0.2–1.2 µs†; on the mixed workload it does 11.6 M commands/s† with p50 / p99 of 91 ns† / 410 ns†.

3.2 What profiling showed

The allocator is the biggest single item, 25–35% of the engine's instructions. Pooling would mean changing M2's containers and move semantics, and the risk wasn't worth it inside L2, so it's a follow-up task (D-044). A glibc tuning probe that changes no code (P1) cut allocator instructions by −8.9%, but wall-clock time stayed within noise — **an honestly reported null result**.

3.3 E1: the story of a complexity bug

- **Symptom**: session throughput fell roughly as 1/n with stream length; 95% of `perf` samples sat in the copy loop of `submit`.
- **Hypothesis**: the commit log grew with an exact `reserve(size + 1)`, so every commit copied the whole history — $O(n^2)$ per session. The L1 event buffer had already been fixed for the same reason (D-022), but M5 didn't follow that rule.

† wall-clock number: partly re-measured locally, origin in App. A

- **Change**: when capacity runs out, `reserve(max(size + 1, 2 × capacity))`, a change of +7 / −2 lines. The reserve still happens before any effect, and nothing allocates after the commit point.
- **Result** (same session, five pairs alternating): non-durable session 4.3–4.6 k commands/s† → 0.71–1.09 M commands/s†; replay latency p50 152 µs† → 0.78 µs†; the durable mode and the engine control stayed within noise.

> **How to say it in an interview**
> The speed-up depends on the stream length, because the bug's cost grows with the square of it, so I don't say "150–250 times† faster". What I say is: the per-command cost of the session went from growing linearly with the number of committed commands to constant, and not a single line of the matching engine or the order book changed.

3.4 Baseline switch, equivalence and memory

- **The baseline stays available as a build option** (D-045): `LOB_BASELINE=ON` compiles a frozen copy of the baseline source file, `OFF` the optimized one, so the optimized code and its generated code are untouched.
- **Equivalence**: on 16 inputs the two implementations produce byte-identical event streams, journals and snapshots.
- **Memory went the opposite way from the estimate**: by capacity the optimized version should use more; measured peak RSS was lower (mixed −1,880 KiB (−5.9%), deep_book −5,464 KiB (−15.2%)). Two mechanisms are verified — the baseline holds old and new blocks at once while growing, and reserved but untouched pages don't count towards RSS — but they don't explain the whole difference, and **I wrote it down as "still unexplained"**.
- One command reproduces the whole comparison: `scripts/l2_reproduce.sh` (73 minutes).

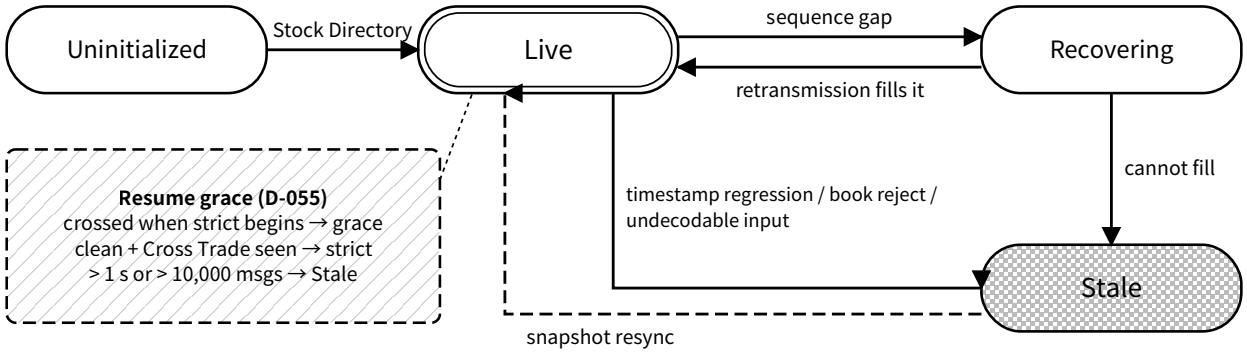
4 L3′ market data and pipeline

4.1 ITCH 5.0 decoding and book reconstruction

- The decoder handles 11 message types (system events, directory, adds, executions, cancels, deletes, replaces, trading actions, cross trades and so on) and skips the other spec types at their exact length. It's zero-copy, and bad input returns an error code instead of crashing.
- **Why not reuse the L1 book** (D-046)? The L1 book can only fill the front of the queue, has no partial cancel and requires increasing sequence numbers. ITCH is the exchange telling you which order traded, and that can be any order. The reconstruction just applies what the exchange says; it never matches.
- `md::Book` **validates before it mutates**: a rejected message leaves the book exactly as it was, so I can assert precisely that a rejection has no side effects.
- Input is read through a fixed buffer instead of `mmap`. Peak memory depends only on configuration — about 47 MB for inputs from 33 MB to 984 MB — and it can read from a pipe, so a file can be downloaded, decompressed and replayed in one stream.

† wall-clock number: partly re-measured locally, origin in App. A
8 / 19

4.2 The FeedHandler state machine



Trading phase (D-054) is orthogonal: never-crossed is enforced only in regular hours while the stock trades

Figure 2. One FeedHandler per instrument. When not Live it publishes no book updates; the resume grace (dashed box) is a sub-state of Live, see chapter 5.

Gaps are detected by **sequence numbers**, not timestamps: quiet periods are normal, so timestamps only have to be non-decreasing. Any input that can't be explained puts that instrument into Stale, and while Stale the downstream only sees state events — it never trades on a book that might be wrong.

4.3 SPSC and the pipeline: multi-threaded but still deterministic

- A single-producer single-consumer ring: indices are 64-bit counters that never wrap, so full and empty need no wasted slot; the producer's and consumer's indices live on separate cache lines; the producer publishes `tail` with release and the consumer reads it with acquire, so seeing a new tail guarantees seeing the whole element.
- **Lossless backpressure** (D-048): when the queue is full the producer spins; nothing is ever dropped.
- **Determinism**: the decode function and the book-thread object are the same code in single-threaded and pipelined mode, and both modes produce byte-identical event streams, books and trading digests (tested under GCC, Clang and TSan).

4.4 Latency waterfall and zero allocation

- Every hop is timestamped with the TSC, calibrated at start-up; the cost of reading the clock twice is measured and subtracted from every sample (D-049). Decode hop p50 / p99 18 ns† / 67 ns†, book hop 12 ns† / 237 ns†, single-threaded replay 9.7 M messages/s†.
- **In saturated mode the "queue hop" is waiting time, not handoff time** (about 1.4 ms† = queue capacity × per-message processing time). To see the cross-core handoff you have to look at the paced mode: p50 104 ns†, with the p99 varying 2.1–40x† between repetitions — virtualization noise, so it's only an upper bound.
- Zero allocation on the hot path: a dedicated test replaces the global `operator new`, each thread arms it at the start of its hot loop, and any allocation fails the test; a deliberately allocating negative control proves it catches them.

† wall-clock number: partly re-measured locally, origin in App. A

4.5 Strategy, risk and simulated fills

- The strategy interface uses CRTP for static dispatch. Against a virtual-function version I measured 10.8† vs 11.2† ns — **no measurable difference**, and I don't claim a gain I couldn't measure.
- Pre-trade risk checks run in a fixed order: kill switch, trading phase, reference price, fat-finger quantity, fat-finger notional, price band, max position, token bucket; one check takes about 6 ns†. Cancels are never blocked, and the kill switch is sticky. The token bucket refills on **market time**, so risk is a pure function of the input.
- The simulated exchange sends approved orders into the **unchanged L1 matching engine**; market executions reported by ITCH become synthetic aggressive orders that trade against our resting orders. Two fill models run side by side — optimistic (we are at the front of the queue) and conservative (we join the back of the visible queue) — and for the same order flow they bound reality from above and below (D-053).

4.6 The CI performance gate counts instructions, not time

On shared CI machines wall-clock time swings by tens of percent; cachegrind's instruction count for a fixed input is exactly reproducible. The gate covers three workloads — decode / book / trading at 33.5 M / 121.3 M / 266.3 M instructions — and fails beyond +2%; it refuses to compare if the compiler or valgrind version differs (D-051). It catches "doing more work", not cache misses, which is why wall-clock benchmarks only run locally.

5 Key story: the resume grace (D-054 / D-055)

5.1 Trading-phase awareness (D-054)

A real exchange's book can **legitimately be locked or crossed** during auctions, halts, LULD pauses and pre- and post-market. The rule: enforce "never crossed" only in regular hours (System Event 'Q' to 'M') while the stock is trading ('T'). Elsewhere a crossed book is allowed, counted and flagged; the strategy stops quoting, and risk rejects new orders without tripping the kill switch.

5.2 Real data broke the rule

I replayed SXTC, which paused twice under LULD on 2019-01-30, from a full day of Nasdaq data. The D-054 rule marked it Stale at the exact moment trading resumed — **it called a correct book wrong**.

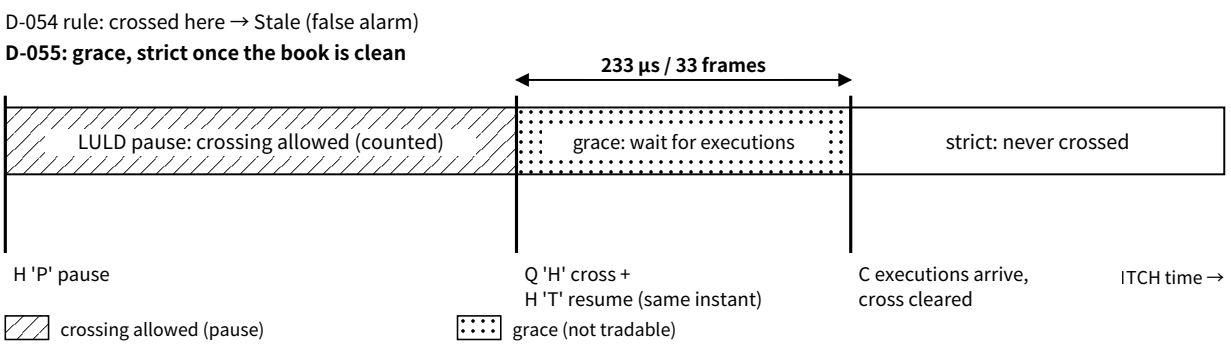


Figure 3. SXTC's first resume (schematic). The reopening cross (Q 'H') and the resume (H 'T') arrive at the same instant; the executions of the orders in the cross arrive afterwards.

† wall-clock number: partly re-measured locally, origin in App. A

10 / 19

The reason is arrival order. During the pause, 10:27:49 to 10:32:49, the book crossed. On resume the exchange runs the cross and announces trading at the same moment, but the executions of the individual orders in that cross arrive 33 frames / 233 µs later (37 frames / 264 µs on the second resume). In between, the book looks crossed.

5.3 The grace rule and its limits

If the book is still crossed when strict hours begin, the feed doesn't go Stale immediately; it enters a **resume grace**:

- during the grace the book stays non-strict, events are flagged and **nothing is tradable**;
- the book is clean **and** a Cross Trade for that stock has been seen (evidence of a reopening cross) → strict mode begins;
- not resolved within 1 second of ITCH time or 10,000 messages of that stock → Stale;
- a new pause during the grace cancels it, and the next resume needs fresh evidence.

Follow-up: couldn't the grace hide a real error?

Three constraints: it needs a Cross Trade as evidence, so a cross without one still goes Stale after a second; it has two limits, time and message count; and nothing trades during the grace. The limits leave about 4,000 times of margin over the measured 233–264 µs, and all 28 graces over three trading days resolved within them. I haven't recorded how long each one took, though, so it's still a parameter to revisit with more data.

5.4 Evidence

- 10 named tests, including a full hand-written event stream in SXTC's real order, "exactly at the limit doesn't time out, one nanosecond more does", a new pause during the grace, and a Cross Trade for another stock not counting.
- The second replay: SXTC stays Live all day, both graces resolved by the executions, and single-threaded and pipelined modes give the same hashes.
- The instruction-count gate stopped the first implementation: checking the timeout inline on the hot path cost the book workload +2.94%; moving it into a non-inlined cold-path function brought it to +1.10%.

How to tell it (about a minute)

From the spec I wrote a reasonable-looking rule: during regular hours the book may never be crossed. Then I replayed real data, and a stock coming back from a pause was flagged as broken at the exact moment it resumed. Instead of just loosening the rule, I looked at the arrival order: the reopening cross and the resume arrive together, and the executions arrive 233–264 µs later. So I designed a bounded grace — it requires evidence of the cross, it has a time and a message limit, and nothing trades during it — and validated it on three full trading days: 26,040 stock-days, not one false alarm. It's my favourite example of measuring first, then making the rule, and writing down why.

† wall-clock number: partly re-measured locally, origin in App. A

6 Market-wide mode and real data

6.1 Market-wide order books (D-056)

- One FeedHandler per stock, all stocks sharing one order pool (order references are unique across the market); stocks are sharded across N worker threads by stock locate modulo N.
- **Why modulo instead of balancing by load?** With modulo, which thread owns which stock depends only on the input, so results are reproducible and comparable. Moving stocks around at run time would depend on scheduling and make determinism and testing much harder.
- The summary hash is the sum of per-stock digests, independent of order, so single-threaded and N-shard runs compare directly.

6.2 Results on real data (Nasdaq 2019-01-30, full day, streamed, never stored)

Item	Result
instruments / final state	8,713, all Live, none Stale
peak live orders	1,742,866
single-threaded vs 4 shards	per-stock hash files byte-identical, summary hash `0xf9bcda01d2d8d01b`
throughput	single-threaded 1.35 M frames/s†, 4 shards 3.11 M frames/s† (more threads than vCPUs, so not linear)
multi-day	26,040 stock-days over three trading days, all Live; all 28 resume graces resolved

A single-stock replay (AAPL): 11.2 GB, 368 million frames, peak memory 98.7 MiB, no Stale.

6.3 Zero allocation in market-wide mode (D-058)

In market-wide mode stocks are created when their directory message arrives, and level arrays grow on demand — both allocate. The fix: one pass writes a profile of each stock's peak number of price levels (the largest is 4,227); before a replay every stock is pre-created with peak × 2x levels and **capped** there. A stock that exceeds it goes Stale; nothing allocates. That takes 45–51 MB, against about 1.8 GB if every stock reserved the maximum. The cost is written down: the profile comes from an earlier day, so a stock might exceed it, and the result is a visible Stale rather than a hidden allocation.

7 L4 order entry and tick-to-trade

7.1 Layering: OUCH and SoupBinTCP

- **SoupBinTCP** only makes sure every message the exchange sends reaches the client, in order and without loss: login, implicit sequence numbers, replay from "next expected sequence" after a reconnect, a 1 second heartbeat, a 15 seconds timeout, end of session. It does **not** guarantee the client's messages.

† wall-clock number: partly re-measured locally, origin in App. A

12 / 19

- **OUCH 4.2** carries the order semantics, and every inbound message can be **safely resent**: the exchange ignores a duplicate token, and a cancel states the size you want left, not a decrement. Because inbound messages are idempotent, the session only needs to guarantee outbound delivery — that is the pivot of the whole recovery design.
- Both layers are **sans-IO** state machines: bytes and the current time go in; bytes to send, events and the next deadline come out. Every transition is tested with hand-written bytes and a fake clock, with no network and no sleeping.

7.2 The order gateway

- Three ids: the strategy's `client_id`; the OUCH token, unique on the wire (a replace gets a new one); and the exchange's order reference number. A token is the date, a per-start instance number and a counter; the instance number is persisted, so a restarted process never reuses a token that day.
- A per-order state machine: PendingNew → Live → Filled / Canceled / Rejected, and fills can still arrive after a cancel was sent, because the two race on the wire. Any report the gateway can't explain — unknown token, overfill, a message after a final state — marks it inconsistent, which trips the kill switch and cancels everything. **I'd rather stop trading than guess the position.**
- Reconnect: pending cancels and replaces are resent unchanged, since they're idempotent; new orders are not resent by default, since they were decided on stale market data. The spec gives no high-water mark for "the replay is over", so I use the spec's own signal: the server only sends a heartbeat after 1 second of silence, so the first heartbeat after login means the replay is complete.

7.3 The exchange simulator

One thread on epoll. The exchange **keeps its own copy of the market book**, because the conservative fill model needs the queue the exchange sees; matching is still the unchanged L1 engine. It implements the spec's four outcomes of a replace, partial cancels, exchange-side throttling (excess new orders and replaces get the spec's reject reason 'x'; **cancels are never throttled**) and cancel-on-disconnect (the cancels are sequenced, so they show up in the replay after reconnecting).

7.4 How I know the network mode is right

When replies arrive depends on thread scheduling, so the same input can interleave differently. I proved two separate things:

† wall-clock number: partly re-measured locally, origin in App. A

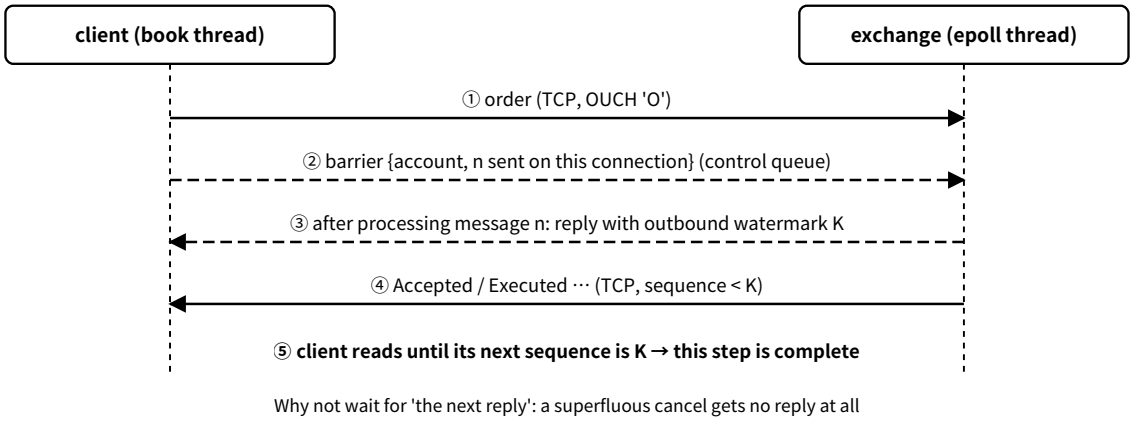


Figure 4. Lockstep mode. The barrier counts inbound messages sent on the current connection; the reply carries the outbound sequence watermark.

- **Lockstep** proves the network stack is transparent: after every step we wait for quiescence, so replies are delivered in exactly the in-process order. The bar is that orders, fills and position hashes are **bit-identical** to in-process — for both fill models, single-threaded and with the multi-threaded pipeline.
- **Free-run** doesn't compare hashes; it checks invariants — exposure within the position limit on both sides, no duplicate token, legal state transitions, and after draining, position, cash, open orders and sequence numbers reconcile. Both sides also **record the interleaving they actually saw, and replaying the recording offline must reproduce it bit for bit**. All of this holds with three injected disconnects and exchange throttling.
- The tests were tested too: removing the lockstep wait makes both lockstep tests fail, and dropping one recorded delivery makes the replay test fail.

7.5 Tick-to-trade

Definition: from the start of decoding the ITCH frame to the OUCH bytes being handed to the kernel (`send()` returns), counting only frames that produced an order, with the clock-read overhead subtracted. **Send first**: market data is forwarded and replies are read only after the frame's orders are out, so no other system call sits inside the window.

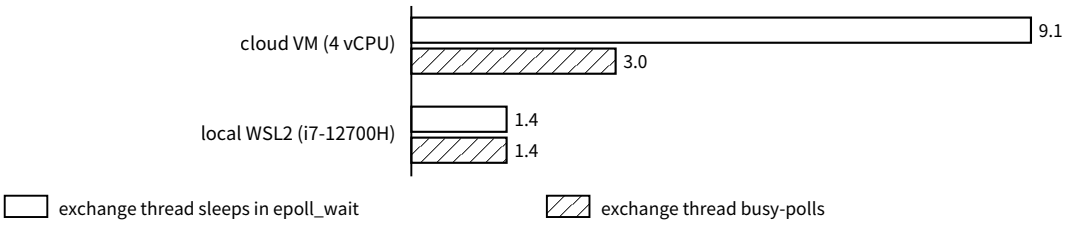


Figure 5. The same loopback measurement on two machines, p50. White: the exchange thread sleeps in epoll_wait; hatched: it busy-polls. Busy-polling helps a lot on the cloud VM and not at all locally.

- In-process, tick → first message of the frame encoded: 0.37 µs† — the software path itself.
- On my machine, over loopback: tick → first message `send()` returned in 1.4 µs†, or 1.4 µs† with a busy-polling exchange thread; one `send()` costs about 0.88 µs†; client `send()` → exchange decoded takes 171 µs†; round trip 447 µs† / 395 µs†.
- **Same experiment, two machines, two conclusions.** On the cloud VM busy-polling cut the first message from 9.1 µs† to 3.0 µs† and the round trip from 28 µs† to 18 µs†, so I

† wall-clock number: partly re-measured locally, origin in App. A

14 / 19

concluded that loopback cost is mostly waking the other thread. On my own machine busy-polling changes nothing, so that conclusion **only holds for the cloud VM**. The method behind it still stands: controlled variables — move everything that isn't on the order path out of the window (reading replies, the eventfd write that forwards market data), use "new order vs first message of the frame" to estimate one `send()`, then test the remaining gap with busy-polling.

- **Why the local round trip is so slow isn't explained yet.** I know it isn't cross-core clock skew (the round trip is timed on the client's own core), it isn't the exchange thread sleeping (busy-polling doesn't change it), and it isn't the software path. The next step is a minimal loopback TCP ping-pong on the same machine, to separate the platform's own loopback latency from my code.
- These aren't wire-to-wire numbers: no NIC, no switch, no real gateway, no kernel bypass. I only compare them with my own previous version.

8 Engineering method

- **CI**: 6 presets with zero warnings, two TSan presets and the cachegrind instruction-count gate; `scripts/check.sh` is the single entry point.
- **Negative controls**: in every sub-stage I deliberately broke the code and confirmed the tests failed, then restored it. Once a negative control reported "not caught"; it turned out the mutant hadn't compiled because of `-Werror`, so the old binary was being tested. Since then I always confirm the build succeeded first.
- **Decision records**: all 65 decisions state the options, the choice, the cost and when to revisit; deviations from a design (for example, a second login for the same account now takes over instead of being rejected) are recorded too.
- **Number discipline**: every number states its environment and source; in this handbook they all come from `numbers.yaml`, and wall-clock numbers carry †.
- **AI collaboration**: AI took part in design and implementation, and `AI_USAGE.md` records the division of work and the author's decisions stage by stage. If asked, I say so plainly and point out that verification rests on tests and reproducible evidence.

9 Limitations and unverified items (with how to answer)

Limitation / not verified	How to answer
wall-clock numbers come from two machines (my WSL2 and a cloud VM); some were only measured in the cloud	"Every number says which machine and date it's from, Appendix A has the sources, and one command re-measures. Re-measuring overturned one of my cloud conclusions, so I don't carry a finding from one machine to another, and I never attribute a change using numbers from different sessions."
the slow local loopback round trip is unexplained	"I know it isn't clock skew and it isn't my software path. Next I'd measure the platform's own loopback with a minimal ping-pong; until then I don't make up an explanation."

† wall-clock number: partly re-measured locally, origin in App. A

Limitation / not verified	How to answer
loopback is not wire-to-wire: no NIC, switch or kernel bypass	"Loopback shows the cost of connecting a user-space program to a kernel socket. Real wire-to-wire needs hardware timestamps on a switch mirror port and PTP on both ends. I only compare against my own previous version."
durability under power loss or an OS crash is not verified	"I tested process kills, truncation at every byte and failed writes. Power loss depends on the file system implementing fsync and rename correctly; I couldn't test that, so I don't claim it."
the L1 engine still allocates when an order rests	"The zero-allocation guarantee covers the market-data hot path. Pooling the engine means changing the frozen L1 code; the risk wasn't worth it, so it's a recorded follow-up."
the fill models don't simulate our own market impact	"Optimistic and conservative give upper and lower bounds for the same order flow. With a closed-loop strategy the totals are no longer ordered — tests found that, and it's in the decision record."
the resume-grace limits need more data	"So far 28 graces over three days all resolved. I haven't recorded the distribution of how long each took, so the limit is a value with margin, not a conclusion."
a restarted process doesn't take over the previous process's live orders	"Tokens never repeat across restarts, that's guaranteed. Taking over old orders needs recognising the old instance by token prefix — it's the clear next step."
the memory difference after E1 isn't fully explained	"Two mechanisms are verified, but they don't account for all of it. I wrote it down as 'still unexplained' instead of inventing a story."

10 Selected interview questions

Q1 Why determinism, and what does it cost? It makes results for the same input comparable: differential tests, single vs multi-threaded, in-process vs network all depend on it. The cost is that I can't use system time or rely on scheduling order — risk refills on market time, and threads only exchange ordered data through SPSC queues.

Q2 Why those memory orders in the SPSC queue? The producer writes the element, then publishes `tail` with release; the consumer reads `tail` with acquire, so seeing the new tail means seeing the whole element. Each side reads its own index relaxed. No third party observes the relative order of the two indices, so seq_cst isn't needed.

Q3 How can multi-threaded output be byte-identical to single-threaded? Both modes run the same decode function and the same book-thread object; the queue is lossless and ordered; state is only ever modified on one thread. Tests compare event-stream and snapshot hashes under GCC, Clang and TSan.

Q4 How do you know the matching engine is correct? Differential testing against a naive reference model on random command streams (20 million commands), an event auditor that checks conservation independently, mutation testing to show the tests catch real bugs, and fault injection at every allocation point.

Q5 What's the best performance fix you made? E1 — not "the engine got faster" but an $O(n^2)$ complexity bug: located by profiling, a written hypothesis, verified by same-session

† wall-clock number: partly re-measured locally, origin in App. A

16 / 19

alternating A/B, with an honest note that the ratio depends on stream length. The other one worth telling is a null result: allocator tuning showed no measurable wall-clock gain, so I didn't adopt it.

Q6 Why does CI gate performance on instruction counts? Wall-clock time on shared machines swings by tens of percent, while instruction counts for a fixed input are exactly reproducible. It catches "more work" regressions, not cache effects, so wall-clock benchmarks run locally. The gate did stop one real regression: the grace timeout check inlined on the hot path.

Q7 Tell me about real data contradicting an assumption. Chapter 5: at a resume the cross and the resume arrive together and the executions 233–264 µs later. The rule called a correct book wrong, so I designed a resume grace with evidence, limits and no trading, and validated it on 26,040 stock-days without a single false alarm.

Q8 Why shard by stock locate modulo N? Ownership depends only on the input, so runs are reproducible and comparable; moving stocks at run time depends on scheduling. The cost is possible imbalance — if hot stocks cluster, I'd build a fixed mapping offline from historical volume, which stays deterministic.

Q9 After a reconnect, how do you know which orders reached the exchange? Log in with the old session and the next expected sequence number, and the exchange replays everything sent while you were gone. Cancels and replaces are resent unchanged because they're idempotent; new orders aren't by default. The first heartbeat after login means the replay is over, so a new order still unanswered at that point never arrived.

Q10 With a network in the middle, timing is nondeterministic — how do you prove correctness? Lockstep proves the network stack is transparent: bit-identical to in-process, using the outbound sequence watermark as a barrier, because a superfluous cancel gets no reply at all. Free-run checks invariants and records both sides' interleaving, and the recording replays bit for bit — so a failure can be brought back and stepped through.

Q11 What's your tick-to-trade, and what does it tell you? Definition first, then numbers: on my machine 0.37 µs† in-process, 1.4 µs† on loopback, 447 µs† round trip. The interesting part is that it overturned my own conclusion: on the cloud VM busy-polling cut loopback from 9.1 µs† to 3.0 µs†, while locally it made no difference — so "it's the wake-up" was a fact about that machine, not a general one. It isn't wire-to-wire, so I only compare it with myself.

Q12 What would you do differently? Bring real data in earlier — rules should be tested against data first; set up controlled-variable measurements like "new order vs first message" from the start; and the next steps are pooling the book's nodes and taking over a previous instance's live orders.

11 Future plans: L5 and L6 (not implemented)

The project scope ends at L4 (D-065). These two are recorded only as future directions:

- **L5 UDP multicast with A/B line arbitration**: the exchange sends the same MoldUDP64 sequenced stream on two independent paths; the receiver deduplicates by message sequence number, takes whichever copy arrives first and delivers strictly in order. When both lines miss a message it waits a bounded window, then asks a retransmission server for the smallest range; if that fails too, the existing recovery state machine (Recovering → Stale) takes over instead of a second recovery mechanism. The bar: for any recoverable

† wall-clock number: partly re-measured locally, origin in App. A

combination of faults, the message sequence handed to the pipeline is byte-identical to reading the file directly. There's a design draft (`docs/design/L5_DESIGN.md`, not implemented).

- **L6 microstructure signal research**: export features from the reconstructed book (columnar files tied to the book hash), test them in and out of sample, and report without claiming trading profits. No design yet.

A Wall-clock numbers at a glance

These are all numbers marked † in this handbook, where they were measured and where they come from; the sources of all other numbers are in `docs/handbook/numbers.yaml`.

Measured on: author WSL2, 2026-10-22

Number	Value	Source
engine throughput (mixed)	11.6 M commands/s	BENCHMARK.md § 10
engine latency p50 (mixed)	91 ns	BENCHMARK.md § 10
engine latency p99 (mixed)	410 ns	BENCHMARK.md § 10
engine p99 (seven profiles)	0.2–1.2 µs	BENCHMARK.md § 11

Measured on: author WSL2, 2026-10-21

Number	Value	Source
before E1: non-durable session throughput	4.3–4.6 k commands/s	DECISIONS.md D-042; BENCHMARK.md § 9
after E1: non-durable session throughput	0.71–1.09 M commands/s	DECISIONS.md D-042; BENCHMARK.md § 9
before E1: replay latency p50	152 µs	BENCHMARK.md § 12
after E1: replay latency p50	0.78 µs	BENCHMARK.md § 12
E1 ratio (depends on stream length)	150–250 times	BENCHMARK.md § 9

Measured on: local machine (Intel i7-12700H, 20 CPUs, WSL2), 2026-11-05

Number	Value	Source
ITCH replay throughput (single thread)	9.7 M messages/s	BENCHMARK.md § 13.1
decode hop p50	18 ns	BENCHMARK.md § 13.1
decode hop p99	67 ns	BENCHMARK.md § 13.1
book hop p50	12 ns	BENCHMARK.md § 13.1
book hop p99	237 ns	BENCHMARK.md § 13.1
cross-thread handoff p50 (paced)	104 ns	BENCHMARK.md § 13.1
one risk check	about 6 ns	BENCHMARK.md § 14.1
CRTP strategy call (ns)	10.8	BENCHMARK.md § 14.1

Number	Value	Source
virtual strategy call (ns)	11.2	BENCHMARK.md § 14.1
in-process tick → first message encoded	0.37 µs	BENCHMARK.md § 15.1
loopback tick → first message send()	1.4 µs	BENCHMARK.md § 15.1
same, exchange thread busy-polls	1.4 µs	BENCHMARK.md § 15.1
round trip tick → Accepted	447 µs	BENCHMARK.md § 15.1
round trip, busy-polling exchange	395 µs	BENCHMARK.md § 15.1
client send() → exchange decoded	171 µs	BENCHMARK.md § 15.1
queue hop, saturated mode	about 1.4 ms	BENCHMARK.md § 13.1
handoff p99 spread over repetitions	2.1–40x	BENCHMARK.md § 13.1
one loopback send()	about 0.88 µs	BENCHMARK.md § 15.1

Measured on: cloud VM 4 vCPU, 2026-11-04

Number	Value	Source
market-wide throughput (single)	1.35 M frames/s	MODULE_NOTES/L3.md
market-wide throughput (4 shards)	3.11 M frames/s	MODULE_NOTES/L3.md

Measured on: cloud VM 4 vCPU, 2026-11-05

Number	Value	Source
cloud: loopback tick → first message send()	9.1 µs	BENCHMARK.md § 15
cloud: same, busy-polling exchange	3.0 µs	BENCHMARK.md § 15
cloud: round trip tick → Accepted	28 µs	BENCHMARK.md § 15
cloud: round trip, busy-polling exchange	18 µs	BENCHMARK.md § 15

B Re-measure locally and regenerate

`remeasure.sh` builds Release, runs `lob_bench`, `lob_itch_bench` and `lob_t2t` on pinned cores, writes the medians, date and machine into `numbers.yaml` and rebuilds; the † footer follows (Appendix A). Run from the repository root; the first time, `sudo apt install python3-venv fonts-noto-cjk fonts-dejavu-core poppler-utils valgrind`:

```
docs/handbook/remeasure.sh                 # measure, rebuild
docs/handbook/remeasure.sh --itch FILE     # + market-wide
docs/handbook/remeasure.sh --l2            # + L2 (~73 min)
docs/handbook/remeasure.sh --from-log LOG  # hand-run log
docs/handbook/build.sh                     # rebuild only
```

† wall-clock number: partly re-measured locally, origin in App. A